Assignment 6

Server.cpp

```
%%writefile server.cpp
#include <iostream>
#include <cstring>
#include <cctype>
#include <algorithm>
#include <sys/socket.h>
#include <netinet/in.h>
#include <unistd.h>

#define PORT 8080
#define MAXLINE 1024

// Data structure to hold menu choice and message payload
struct Packet {
    int choice;
    char text[MAXLINE];
};

// Function to change case (Uppercase <-> Lowercase)
std::string changeCase(std::string str) {
    for (char &c : str) {
        if (std::isupper(c)) c = std::tolower(c);
        else if (std::islower(c)) c = std::toupper(c);
    }
    return str;
}

// Function to reverse the string
std::string reverseString(std::string str) {
    std::reverse(str.begin(), str.end());
    return str;
}

int main() {
```

```
    int sockfd;
    struct sockaddr_in servaddr, cliaddr;

    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket creation failed");
        return -1;
    }

    memset(&servaddr, 0, sizeof(servaddr));
    memset(&cliaddr, 0, sizeof(cliaddr));

    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = INADDR_ANY;
    servaddr.sin_port = htons(PORT);

    if (bind(sockfd, (const struct sockaddr *)&servaddr, sizeof(servaddr))
< 0) {
        perror("Bind failed");
        return -1;
    }

    std::cout << "[SERVER] UDP Server running on port " << PORT <<
"...\n";

    while (true) {
        Packet pkt;
        socklen_t len = sizeof(cliaddr);

        int n = recvfrom(sockfd, &pkt, sizeof(pkt), MSG_WAITALL, (struct
sockaddr *)&cliaddr, &len);
        if (n <= 0) continue;

        if (pkt.choice == 3) {
            std::cout << "[SERVER] Client requested exit. Closing
connection.\n";
            std::string exitMsg = "Session ended by server.";
            sendto(sockfd, exitMsg.c_str(), exitMsg.length(), MSG_CONFIRM,
(const struct sockaddr *)&cliaddr, len);
            break;
        }
```

```
        std::cout << "[SERVER] Choice Received: " << pkt.choice << " |
Input Text: \"" << pkt.text << "\"\n";

        std::string result;
        if (pkt.choice == 1) {
            result = changeCase(pkt.text);
        } else if (pkt.choice == 2) {
            result = reverseString(pkt.text);
        } else {
            result = "Invalid Option";
        }

        sendto(sockfd, result.c_str(), result.length(), MSG_CONFIRM,
(const struct sockaddr *)&cliaddr, len);
    }

    close(sockfd);
    return 0;
}
```

Client.cpp

```
%%writefile client.cpp
#include <iostream>
#include <cstring>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>

#define PORT 8080
#define MAXLINE 1024

struct Packet {
    int choice;
    char text[MAXLINE];
};
```

```
int main() {
    int sockfd;
    struct sockaddr_in servaddr;

    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket creation failed");
        return -1;
    }

    memset(&servaddr, 0, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_port = htons(PORT);
    servaddr.sin_addr.s_addr = inet_addr("127.0.0.1");

    while (true) {
        std::cout << "\n MENU\n";
        std::cout << "1. Change Case\n";
        std::cout << "2. Reverse String\n";
        std::cout << "3. Exit\n";
        std::cout << "Enter your choice (1-3): ";

        Packet pkt;
        if (!(std::cin >> pkt.choice)) break;
        std::cin.ignore(); // Flush newline character from buffer

        if (pkt.choice == 3) {
            memset(pkt.text, 0, sizeof(pkt.text));
            sendto(sockfd, &pkt, sizeof(pkt), MSG_CONFIRM, (const struct
sockaddr *)&servaddr, sizeof(servaddr));
            std::cout << "[CLIENT] Exiting client application.\n";
            break;
        }

        if (pkt.choice != 1 && pkt.choice != 2) {
            std::cout << "Invalid selection! Please choose 1, 2, or 3.\n";
            continue;
        }

        std::cout << "Enter string: ";
        std::string inputStr;
```

```
        std::getline(std::cin, inputStr);

        strncpy(pkt.text, inputStr.c_str(), sizeof(pkt.text) - 1);
        pkt.text[sizeof(pkt.text) - 1] = '\0';

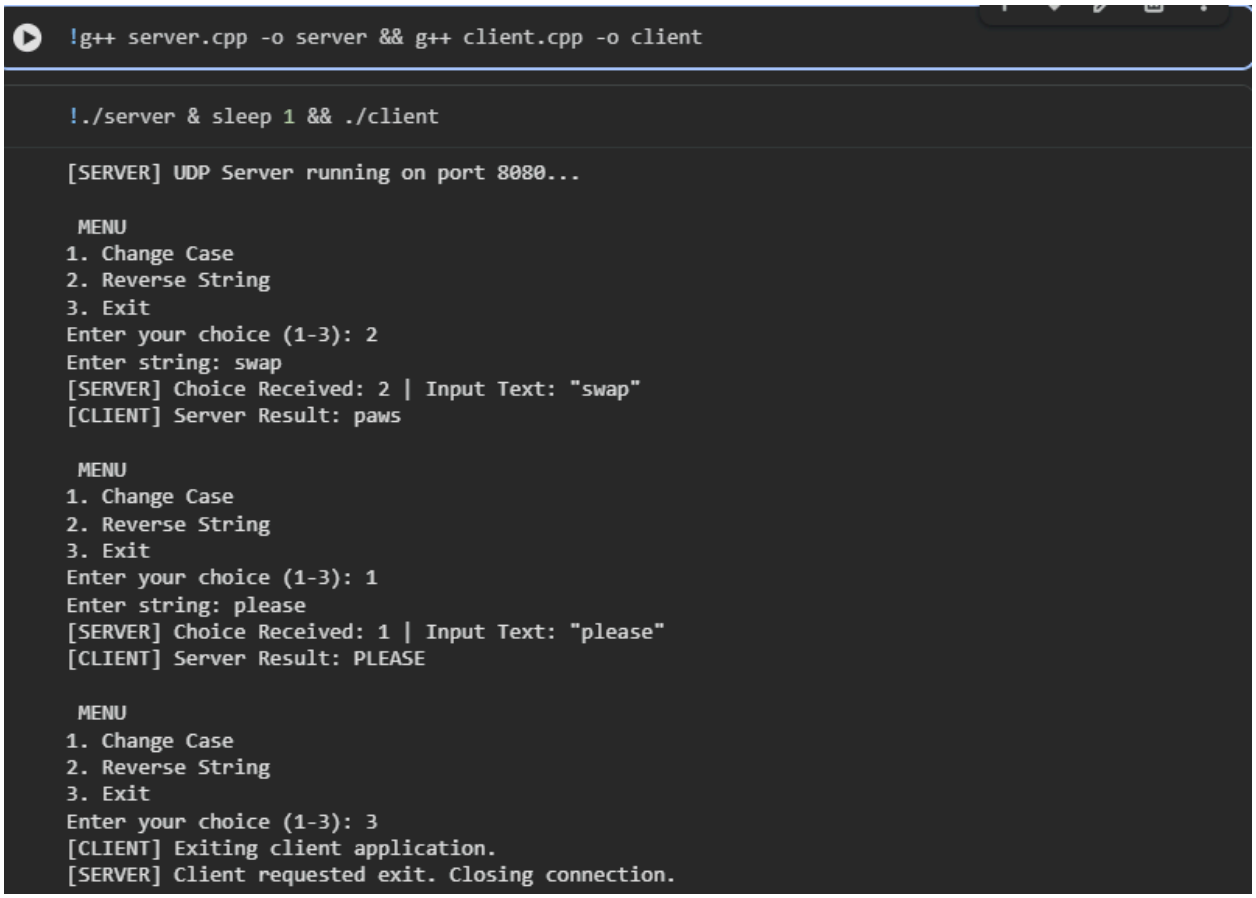
        sendto(sockfd, &pkt, sizeof(pkt), MSG_CONFIRM, (const struct
sockaddr *)&servaddr, sizeof(servaddr));

        char buffer[MAXLINE];
        socklen_t len = sizeof(servaddr);
        int n = recvfrom(sockfd, buffer, MAXLINE - 1, MSG_WAITALL, (struct
sockaddr *)&servaddr, &len);
        buffer[n] = '\0';

        std::cout << "[CLIENT] Server Result: " << buffer << "\n";
    }

    close(sockfd);
    return 0;
}
```

OUTPUT



```
!g++ server.cpp -o server && g++ client.cpp -o client

!./server & sleep 1 && ./client

[SERVER] UDP Server running on port 8080...

 MENU
1. Change Case
2. Reverse String
3. Exit
Enter your choice (1-3): 2
Enter string: swap
[SERVER] Choice Received: 2 | Input Text: "swap"
[CLIENT] Server Result: paws

 MENU
1. Change Case
2. Reverse String
3. Exit
Enter your choice (1-3): 1
Enter string: please
[SERVER] Choice Received: 1 | Input Text: "please"
[CLIENT] Server Result: PLEASE

 MENU
1. Change Case
2. Reverse String
3. Exit
Enter your choice (1-3): 3
[CLIENT] Exiting client application.
[SERVER] Client requested exit. Closing connection.
```